



Architekturdokument

in Phase 3 aktualisiert

Projekttitel: Network Architect

Kurs: Software Engineering (DLMCSPSE01_D)

Student: Johannes Öllerer

Matrikelnummer: 92211477

Datum: 01.August 2026



Inhaltsverzeichnis

1 Technologieübersicht.....	4
1.1 Programmiersprache - Python	4
1.2 GUI-Framework - PySide6	4
1.3 Persistenz (Datenbank) - SQLite	4
1.4 Testing-Framework - pytest	5
1.5 Deployment/Packaging - PyInstaller	5
1.6 Versionskontrolle - GitHub.....	5
2 Architekturmodell	5
3 Struktur.....	6
3.1 Model Layer	6
3.1.1 GameSession	6
3.1.2 Level	7
3.1.3 Network.....	8
3.1.4 Node	9
3.1.5 Bridge	9
3.1.6 NodeType-Eigenschaften	10
3.1.7 BridgeType-Eigenschaften.....	10
3.1.8 NodeConfig-Eigenschaften.....	11
3.2 ViewModel Layer.....	12
3.2.1 LevelSelectionViewModel	12
3.2.2 GameViewModel	13
3.2.3 StatisticsViewModel	15
3.3 View Layer.....	16
3.3.1 MainMenuView	16
3.3.2 LevelSelectionView	17
3.3.3 GameView	19
3.3.4 StatisticsView	21
3.3.5 Zusätzliche UI-Komponenten	22
3.4 Persistence Layer	25
3.5 UML-Klassendiagramm MVVM.....	26
4 Verhalten - Sequenzdiagramm Brücke platzieren	28

4.1 BridgeType-Auswahl	28
4.2 FromNode, GridPoints und ToNode Auswahl	28
4.3 Bridgeplacement - Validierung und Platzierung.....	28
4.4 Architektur-Pattern.....	29
4.5 Performance- und Redundanz-Bewertung.....	29
5 Verzeichnisse.....	30
5.1 Tabellenverzeichnis	30
5.2 Abbildungsverzeichnis.....	30

1 Technologieübersicht

Dieses Kapitel beschreibt die zentralen Technologien für die Entwicklung von Network Architect und begründet deren Auswahl unter Berücksichtigung der Projektvorgaben. Die Technologieentscheidungen priorisieren bewährte, dokumentierte Komponenten, die eine klare Trennung von View, ViewModel und Model ermöglichen sowie Risiken wie Dependency-Kompatibilität (R1) und UI-Komplexität (R4) adressieren. Folgend werden Programmiersprache, GUI-Framework, Persistenz, Testing und Deployment detailliert.

1.1 Programmiersprache - Python

Python erfüllt als moderne objektorientierte Hochsprache die explizite Vorgabe der Aufgabenstellung und ist ideal für die Entwicklung einer Windows 10/11 Desktop-Anwendung mit grafischer Benutzeroberfläche.

- Standalone Deployment: PyInstaller erzeugt executable ohne zusätzliche Laufzeitumgebungen über "Standardbibliotheken" hinaus.
- GUI-fähig: PySide6 ermöglicht "selbsterklärende, intuitive GUI" für interaktive Netzwerk-Visualisierung mit click-basierter Bridge-Platzierung.
- Testing: Unterstützt Unit-Tests für Qualitätssicherung.

1.2 GUI-Framework - PySide6

PySide6 ermöglicht als GUI-Library die Vorgaben der Aufgabenstellung einer "selbsterklärenden, intuitiven grafischen Benutzeroberfläche" zu erfüllen.

- Komplexe GUI: QGraphicsView für interaktive Netzwerk-Visualisierung (Nodes/Bridges) mit click-basierter Bridge-Platzierung
- Standalone-fähig: PyInstaller inkludiert alle Qt-Dependencies, keine zusätzlichen Installationen erforderlich
- MVVM-fähig: Signals/Slots ermöglichen saubere View-Model-Trennung für Architekturdokument

1.3 Persistenz (Datenbank) - SQLite

SQLite erfüllt als lokale, serverlose Datenbank die Vorgabe der Aufgabenstellung einer Windows 10/11 Standalone-Anwendung ohne zusätzliche Softwarekomponenten.

- Null-Konfiguration: Keine Installation/Server
- Transaktional: Sichert GameSessions/Levels gegen Korruption bei Abstürzen
- Standardbibliothek: Python sqlite3 built-in Standardbibliothek
- Performance: Schnelle CRUD für Spielstände

1.4 Testing-Framework - pytest

pytest wurde als Test-Framework gewählt, um die Qualitätssicherung durch Durchführung und Dokumentation von Testfällen sicherzustellen.

- Testprotokoll: Automatische Berichte mit Testfall-ID, Vorbedingungen, Eingaben, erwartetes/tatsächliches Ergebnis
- Branch-Coverage: Vollständige Abdeckung aller Spielregeln (Budgetprüfung, Netzwerk-Topologie, Node-Kapazitäten) durch automatisierte Tests
- Fixtures: Wiederverwendbare Testdaten für Levels/GameSessions

1.5 Deployment/Packaging - PyInstaller

PyInstaller erzeugt eine vollständig eigenständige .exe-Datei für Windows 10/11, die ohne Python-Installation oder zusätzliche Komponenten direkt ausführbar ist.

- Ein-Klick-Deployment: ".exe im App-Paket" – Benutzer startet direkt, keine Python-Installation nötig
- Komplettes Packaging: PySide6/SQLite-Dependencies automatisch eingebettet
- Portable: Funktioniert auf jedem Windows 10/11 PC ohne Registry/Umwgebungsvariablen

1.6 Versionskontrolle - GitHub

GitHub wurde in der Aufgabenstellung als Plattform zur Versionsverwaltung, vorgeschrieben.

- Pflichtkonform
- Versionshistorie

2 Architekturmodell

Network Architect verwendet die MVVM-Architektur (Model-View-ViewModel), um View (GUI) von Geschäftslogik (Model) vollständig zu trennen und bietet Entkopplung (GUI-Änderungen berühren nie Model), Testbarkeit (Spielregeln unit-testbar ohne GUI-Startup), Wartbarkeit (zentralisierte Binding-Logik) und Skalierbarkeit (neue Views ohne Model-Änderung). Für Network Architect ermöglicht dies parallele Entwicklung (GUI + Logik getrennt), 100% Testabdeckung von Geschäftsregeln und klare Klassenbeziehungen für das geforderte UML-Klassendiagramm.

3 Struktur

3.1 Model Layer

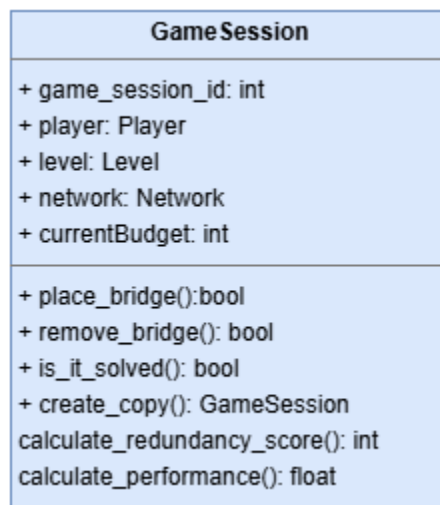
Die wichtigsten Klassen wurden bereits im Spezifikationsdokument definiert. Diese sind `GameSession`, `Level`, `Network`, `Node` und `Bridge`. Zusätzlich zu diesen Klassen müssen noch weitere Klassen definiert werden.

- `NodeType`
- `BridgeType`
- `NodeConfig`

3.1.1 GameSession

In der Klasse `GameSession` werden die Spielablauf-Regeln GR-15 bis GR-17 verankert. Sie stellt sicher, dass der Fortschritt einer Session nach erfolgreichem Levelabschluss automatisch gespeichert wird und damit über mehrere Spielsitzungen hinweg persistent bleibt (GR-15/GR-16 im Zusammenspiel mit `Level/Network` und Persistenz). Außerdem koordiniert sie den Status eines Levels innerhalb einer Session und triggert nach erfolgreicher Lösung bzw. bei Regelverstößen die passenden Reaktionen der Anwendung, womit sie an GR-17 (kein Fortschritt bei ungültigen Aktionen) beteiligt ist.

Abbildung 1: UML-Klasse `GameSession`

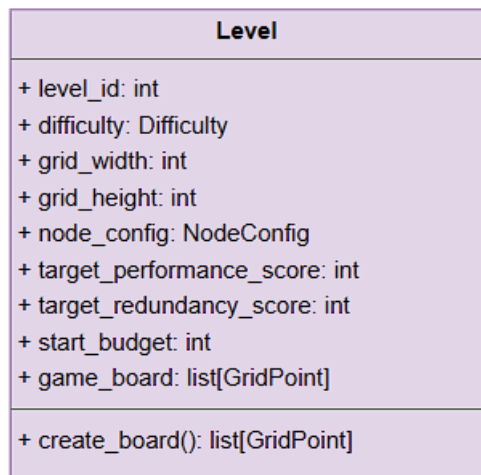


Quelle: Erstellt mit draw.io

3.1.2 Level

In der Klasse Level sind vor allem die Budget- und Rahmenregeln für ein Puzzle verankert, insbesondere GR-01 (jedes Level startet mit einem fest definierten Budget) über das Attribut maxBudget und die zugehörige Initialisierung. Außerdem definiert Level über Rastergröße, Schwierigkeit und Zielscore die Rahmenbedingungen, wann ein Level als gelöst gilt, und bildet damit die Grundlage für die Anwendung der Bewertungsregeln GR-13 und GR-14 sowie für den Ablauf bei Levelabschluss nach GR-15 und GR-16.

Abbildung 2: UML-Klasse Level

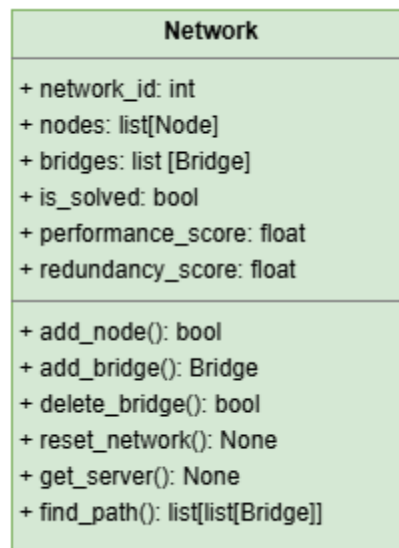


Quelle: Erstellt mit draw.io

3.1.3 Network

Die Klasse Network verwaltet alle Nodes und Bridges eines Levels und ist zentrale Stelle für die Topologie-Validierung nach den Geschäftsregeln GR-05 bis GR-07. Beim Platzieren einer Bridge erhält sie neben den beteiligten Nodes auch den Pfad über die Rasterpunkte und prüft, ob diese Führung zulässig ist, keine verbotenen Kreuzungen oder Kollisionen entstehen, alle Verbindungskapazitäten eingehalten werden und das resultierende Netzwerk weiterhin die Erreichbarkeits- und Redundanzanforderungen erfüllt; zusätzlich berechnet Network daraus die Performance- und Redundanzwerte gemäß GR-13 und GR-14.

Abbildung 3: UML-Klasse Network

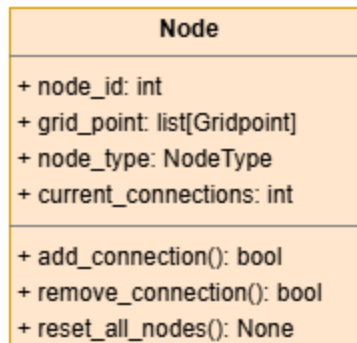


Quelle: Erstellt mit draw.io

3.1.4 Node

In der Klasse Node sind die Node-Spezialisierungsregeln GR-08 bis GR-12 verankert. Über den NodeType und die Attribute zu aktuellen und maximalen Verbindungen stellt Node sicher, dass es pro Level genau einen Server gibt, Server mindestens zwei Verbindungen haben, Firewalls maximal zwei Verbindungen besitzen und die zulässige Verbindungsanzahl je nach Typ nicht überschritten wird (GR-08 bis GR-10, GR-12). Zudem spiegelt Node die Regel wider, dass ein Node seinen Typ während des Spiels nicht ändern darf, indem der NodeType nach der Initialisierung unverändert bleibt (GR-11).

Abbildung 4: UML-Klasse Node

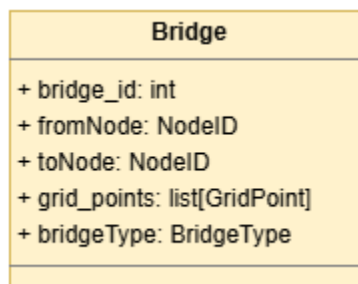


Quelle: Erstellt mit draw.io

3.1.5 Bridge

In der Klasse Bridge sind vor allem die Regeln zu Kosten, Budgetauswirkung und zulässigen Verbindungen verankert, insbesondere GR-03 und GR-04. Jede Bridge trägt ihren BridgeType mit festen Kosten und Bandbreite, sodass beim Platzieren die entsprechenden Kosten vom Level-Budget abgezogen und beim Entfernen wieder gutgeschrieben werden können (GR-03, GR-04) und gleichzeitig die Verbindungskapazitäten der beteiligten Nodes gemäß GR-06 und GR-10 berücksichtigt werden.

Abbildung 5: UML-Klasse Bridge



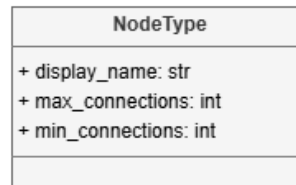
Quelle: Erstellt mit draw.io

3.1.6 NodeType-Eigenschaften

Die Klasse NodeType (siehe Abbildung 6) ist eine Enumeration und kann folgende Werte einnehmen.

- SERVER
- CLIENT
- FIREWALL
- ROUTER

Abbildung 6: UML-Klasse NodeType



Quelle: Erstellt mit draw.io

Die Eigenschaften der verschiedenen Nodes sind wie folgt definiert. Die Werte für minConnections und maxConnections sind an die Geschäftsregeln GR-08 bis GR-10 angelehnt. Die minConnections sind bei den NodeTypeS CLIENT, FIREWALL und ROUTER zwar technisch auf 0 gesetzt, jedoch gilt das Netzwerk erst als vollständig, wenn alle Nodes mit dem Server verbunden sind (GR-05). Dadurch muss jeder Node mindestens eine Verbindung besitzen, um das Level erfolgreich abzuschließen – die Validierung erfolgt nicht über die NodeType-Eigenschaft, sondern über die Topologie-Prüfung in is_it_solved().

Tabelle 1: NodeType-Eigenschaften

NodeType	minConnections	maxConnections	Besonderheiten
SERVER	2	4	Exakt 1 pro Level
CLIENT	0	4	
FIREWALL	0	2	max 2 Verbindungen
ROUTER	0	4	

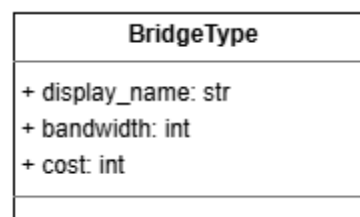
Quelle: Eigene Darstellung

3.1.7 BridgeType-Eigenschaften

Die Klasse BridgeType (siehe Abbildung 7) ist eine Enumeration und kann folgende Werte einnehmen.

- ETHERNET
- WiFi 2.4G
- WiFi 5G
- FIBER

Abbildung 7: UML-Klasse BridgeType



Quelle: Erstellt mit draw.io

Die Eigenschaften der verschiedenen Bridges sind wie folgt definiert. Die festen Kosten und Bandbreiten pro BridgeType werden zur Umsetzung der Budget- und Performance-Regeln (GR-01 bis GR-04, GR-13) verwendet.

Tabelle 2: BridgeType-Eigenschaften

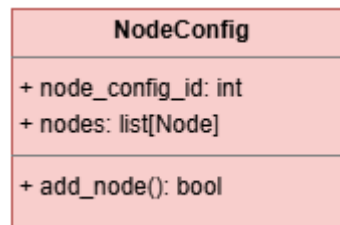
BridgeType	bandwidth (Mbit/s)	cost (€/Segment)	Besonderheiten
ETHERNET	100	10	Kabelverbindung
WIFI_24G	300	20	nur MOBILE/WIFI_AP Verbindung zulässig
WIFI_5G	600	35	nur MOBILE/WIFI_AP Verbindung zulässig
FIBER	2000	120	Highspeed-Backbone

Quelle: Eigene Darstellung

3.1.8 NodeConfig-Eigenschaften

Die Klasse NodeConfig (siehe Abbildung 8) speichert die Startkonfiguration eines Levels, d.h. alle Nodes mit Position und NodeType, um Level-Reset gemäß GR-16 zu ermöglichen. Jedes Level besitzt genau eine NodeConfig, aus der beim Start ein leeres Network mit allen Nodes erzeugt wird.

Abbildung 8: UML-Klasse NodeConfig



Quelle: Erstellt mit draw.io

3.2 ViewModel Layer

Network Architect implementiert drei ViewModels zur Steuerung der Hauptfunktionen: LevelSelectionViewModel für Spieler- und Levelauswahl, GameViewModel für das Gameplay und StatisticsViewModel für die Anzeige von Spielstatistiken. Alle ViewModels erben von QObject und nutzen PySide6's Signal/Slot-Mechanismus für reaktive View-Updates.

3.2.1 LevelSelectionViewModel

Das LevelSelectionViewModel verwaltet die Auswahl von Spieler und Level sowie die Erstellung neuer Spielerprofile. Es lädt alle verfügbaren Spieler und freigeschaltete Levels aus der Datenbank und erzeugt daraus die Startkonfiguration für eine neue GameSession.

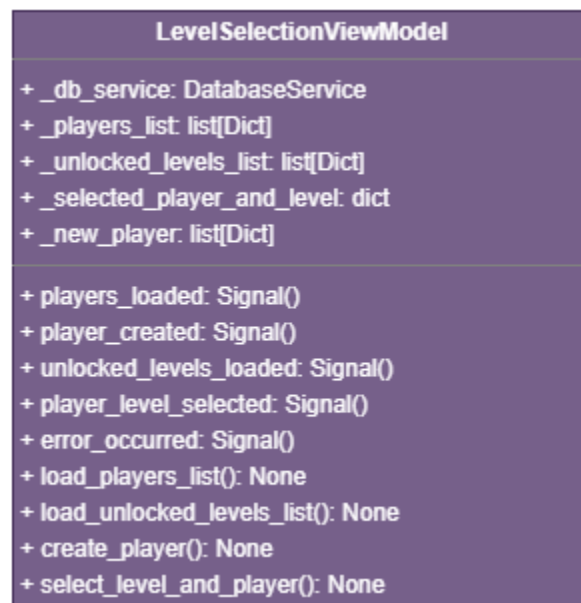
Properties:

- `_db_service`: DatabaseService Verbindung zur Datenbank
- `_players_list`: list[Dict] - Liste aller Spieler
- `_unlocked_levels_list`: list[Dict] - Freigeschaltete Levels
- `_selected_player_and_level`: dict - Ausgewähltes Level und Spieler
- `new_player`: list[Dict] - Neu erstellter Spieler

Methoden:

- `load_players_list()` - Lädt alle Spieler aus DatabaseService
- `load_unlocked_levels_list()` - Lädt freigeschaltete Levels für Spieler
- `create_player()` - Erstellt neuen Spieler
- `select_level_and_player()` - Validiert und speichert Auswahl

Abbildung 9: UML-Klasse LevelSelectionViewModel



Quelle: Erstellt mit draw.io

3.2.2 GameViewModel

Das GameViewModel kapselt das laufende Spiel: Es hält Referenzen auf GameSession, Level und Network und stellt alle relevanten Werte für die GameView bereit. Zudem koordiniert es alle Spielaktionen, indem es Bridge-Platzierungen mit Knoten-IDs und Rasterpunkten an das Network weitergibt, Budget- und Regelprüfungen nach GR-01 bis GR-17 ausführen lässt und die angezeigten Statuswerte wie Budget, Performance und Redundanz aktualisiert.

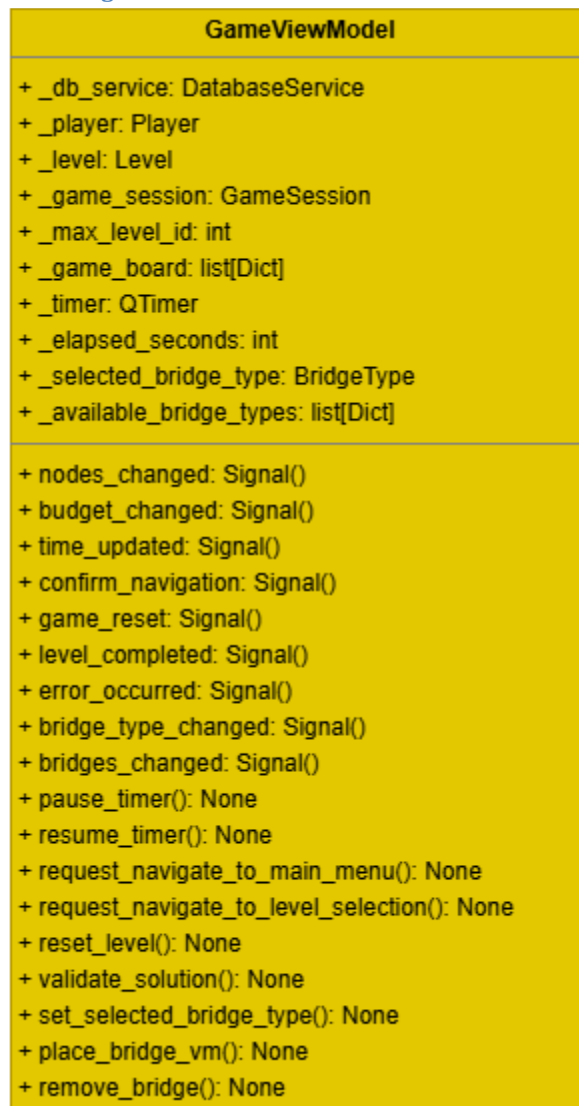
Properties:

- `_db_service`: DatabaseService Verbindung zur Datenbank
- `_player`: Player - aktueller Spieler
- `_level`: Level - aktuelles Level
- `_game_session`: GameSession - aktuelle GameSession
- `_max_level_id`: int - ID des letzten Level
- `_game_board`: list[Dict] - Liste aller GridPoints
- `_timer`: QTimer - Timer für Spielzeit-Tracking
- `_elapsed_time`: int - Verstrichene Spielzeit
- `_selected_bridge_type`: BridgeType - Name des ausgewählten Bridge-Typs
- `_available_bridge_types`: list[Dict] - Verfügbare Bridge-Typen

Methoden:

- `pause_timer()` - Pausiert den Spielzeit-Timer
- `resume_timer()` - Setzt den Timer fort
- `request_navigate_to_main_menu()` - Fordert Hauptmenü an
- `request_navigate_to_level_selection()` - Fordert Levelauswahl an
- `reset_level()` - Setzt Level auf Startzustand zurück
- `validate_solution()` - Prüft ob Level gelöst ist
- `set_selected_bridge_type()` - Wählt Bridge-Typ aus
- `place_bridge_vm()` - Platziert Bridge
- `remove_bridge()` - Entfernt Bridge

Abbildung 10: UML-Klasse GameViewModel



Quelle: Erstellt mit draw.io

3.2.3 StatisticsViewModel

Das StatisticsViewModel lädt und verwaltet Spielstatistiken aus der Datenbank. Es stellt Dropdown-Listen für Spieler- und Levelauswahl bereit und lädt auf Anfrage die vollständigen Statistiken für einen ausgewählten Spieler (alle gelösten Levels) oder ein ausgewähltes Level (alle Spieler, die es gelöst haben).

Properties:

- `_db_service`: DatabaseService Verbindung zur Datenbank
- `_player_statistics`: list[Dict] - Abgeschlossene Levels eines Spielers
- `_level_statistics`: list[Dict] - Alle Spieler, die ein Level gelöst haben
- `_available_players`: list[Dict] - Alle Spieler {id, name} für Dropdown
- `_available_levels`: list[Dict] - Alle Levels {id, difficulty} für Dropdown

Methoden:

- `load_available_players()` - Lädt alle Spieler aus Datenbank
- `load_available_levels()` - Lädt alle Levels aus Datenbank
- `load_player_statistics(player_id: int)` - Lädt abgeschlossene Levels für Spieler
- `load_level_statistics(level_id: int)` - Lädt alle Spieler, die ein Level abgeschlossen haben

Abbildung 11: UML-Klasse StatisticsViewModel



Quelle: Erstellt mit draw.io

3.3 View Layer

Network Architect implementiert vier Hauptviews zur Darstellung der Benutzeroberfläche: MainMenuView als Startbildschirm, LevelSelectionView für Spieler- und Levelauswahl, GameView als zentraler Spielbildschirm und StatisticsView zur Anzeige von Spielstatistiken. Alle Views erben von QWidget und nutzen PySide6's Signal-Mechanismus für Navigation und Benutzerinteraktionen.

3.3.1 MainMenuView

Die MainMenuView ist der erste Bildschirm nach dem Programmstart und fungiert als zentrales Hauptmenü der Anwendung. Sie bietet drei Navigationsoptionen über Buttons und besitzt bewusst kein dediziertes ViewModel, da sie ausschließlich Navigationssignale emittiert und keine Geschäftslogik oder Datenverwaltung benötigt. Die MainMenuView erbt von QWidget und kommuniziert über Signals mit dem MainWindow, das den Navigation-Flow über QStackedWidget steuert. Als einzige View ohne ViewModel folgt sie dem einfachen Observer-Pattern für reine Navigations-Aktionen.

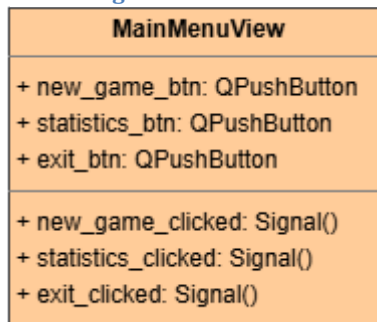
UI-Komponenten:

- new_game_btn (QPushButton): - Startet Levelauswahl-Screen
- statistics_btn (QPushButton): - Öffnet Statistik-Screen
- exit_btn (QPushButton): - Beendet die Anwendung

Signals:

- new_game_clicked: - Emittiert beim Klick auf "New Game"
- statistics_clicked: - Emittiert beim Klick auf "Statistic"
- exit_clicked: - Emittiert beim Klick auf "Exit"

Abbildung 12: UML-Klasse MainMenuView



Quelle: Erstellt mit draw.io

3.3.2 LevelSelectionView

Die LevelSelectionView ermöglicht Spieler- und Levelauswahl sowie die Erstellung neuer Spielerprofile. Sie ist an das LevelSelectionViewModel gebunden und zeigt dynamisch freigeschaltete Levels basierend auf dem gewählten Spieler. Die View implementiert reaktive UI-Updates durch Signal/Slot-Binding mit dem ViewModel. Die LevelSelectionView bindet sich beim Initialisieren an vier ViewModel-Signals für automatische UI-Updates nach Datenbank-Operationen.

UI-Komponenten:

- Bereich Spielerauswahl:
 - player_combo (QComboBox): - Dropdown zur Auswahl vorhandener Spieler
 - new_player_btn (QPushButton): - Button zum Erstellen neuer Spieler
 - QInputDialog: - Popup-Dialog zur Eingabe des Spielernamens
- Bereich Levelauswahl:
 - levels_list (QListWidget): - Liste der freigeschalteten Levels mit Schwierigkeitsgrad
 - start_btn (QPushButton): - Startet ausgewähltes Level (deaktiviert bis Level gewählt)
 - back_btn (QPushButton): - Zurück zum Hauptmenü

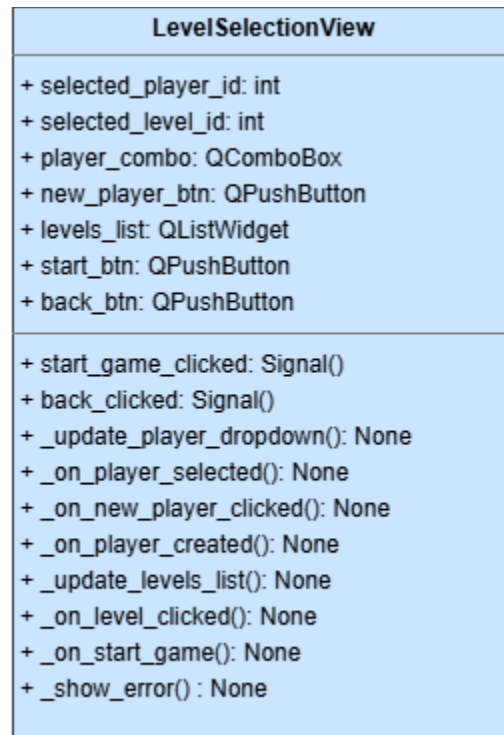
State-Management:

- selected_player_id (int): - ID des ausgewählten Spielers
- selected_level_id (int): - ID des ausgewählten Levels

Signals:

- start_game_clicked (int, int): - Emittiert player_id und level_id beim Spielstart
- back_clicked: - Emittiert bei Navigation zurück zum Hauptmenü

Abbildung 13: UML-Klasse LevelSelectionView



Quelle: Erstellt mit draw.io

3.3.3 GameView

Die GameView ist der zentrale Spielbildschirm mit 3-Panel-Layout (Links: Controls, Mitte: Spielfeld, Rechts: Bridge-Auswahl). Sie visualisiert das aktuelle Network mit Nodes und Bridges und ermöglicht click-basierte Bridge-Platzierung über ein QGraphicsScene/QGraphicsView-System. Die View ist an das GameViewModel gebunden und nutzt reaktive Updates über PySide6-Signals. Die GameView implementiert vollständige MVVM-Trennung: Alle Geschäftslogik-Operationen (Validierung, Budget-Berechnung, Score-Kalkulation) erfolgen im GameViewModel bzw. Model Layer. Die View ist rein präsentativ und transformiert nur Benutzer-Eingaben in ViewModel-Kommandos.

UI-Komponenten:

- Linke Sidebar (Controls):
 - main_menu_btn (QPushButton): - Navigation zum Hauptmenü
 - level_select_btn (QPushButton): - Navigation zur Levelauswahl
 - reset_btn (QPushButton): - Setzt Level auf Startzustand zurück
 - validate_btn (QPushButton): - Prüft Lösung und berechnet Scores (GR-13/GR-14)
 - help_btn (QPushButton): - Zeigt Hilfe-Dialog mit Spielregeln
 - time_label (QLabel): - Echtzeit-Timer im Format "mm:ss"
- Zentrales Spielfeld:
 - game_canvas (GameCanvas): - Custom QGraphicsView für interaktive Darstellung
 - scene (QGraphicsScene): - Container für alle grafischen Elemente
 - coord_mapper (CoordinateMapper): - Grid-zu-Pixel-Transformation
- Rechte Sidebar (Bridge-Auswahl):
 - budget_label (QLabel): - Aktuelles Budget
 - bridge_type_buttons (dict[QPushButton]): - Checkable Buttons pro BridgeType
 - bridge_place_btn (QPushButton): - Bestätigt Bridge-Platzierung
 - bridge_delete_btn (QPushButton): -Aktiviert Lösch-Modus

State-Management:

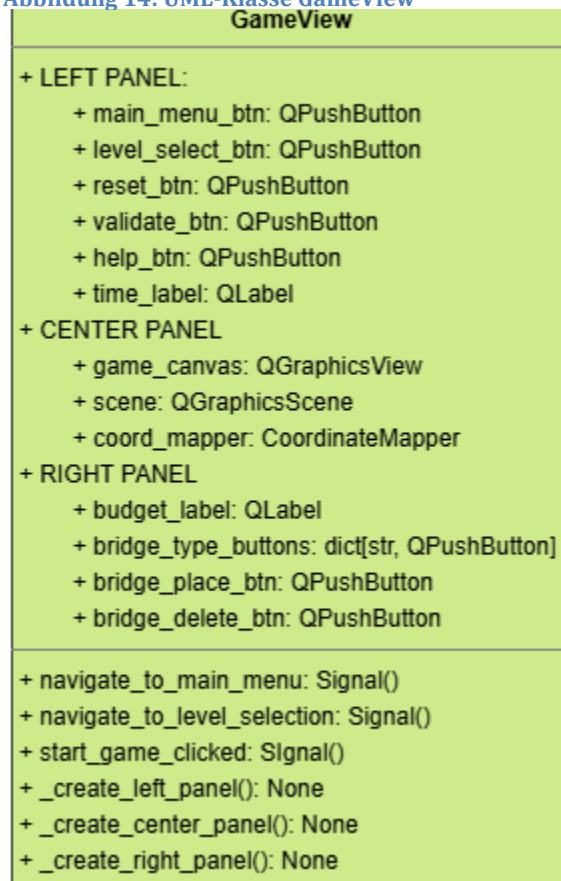
- placement_active (bool): - Status ob Platzierung läuft
- placement_from_node_id (int): - ID des Start-Nodes

- placement_grid_points (list[int]): - Gesammelte GridPoint-IDs
- _selected_bridge_type (BridgeType): - Aktuell gewählter Typ

Signals:

- navigate_to_main_menu: - Emittiert nach bestätigter Navigation zum Hauptmenü
- navigate_to_level_selection: - Emittiert nach bestätigter Navigation zur Levelauswahl
- start_game_clicked (int, int): - Emittiert player_id/level_id für nächstes Level

Abbildung 14: UML-Klasse GameView



Quelle: Erstellt mit draw.io

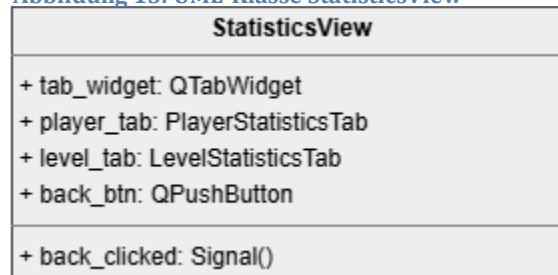
3.3.4 StatisticsView

Die StatisticsView zeigt Spielstatistiken in zwei Tabs (PlayerStatisticsTab: alle gelösten Levels eines Spielers; LevelStatisticsTab: alle Spieler, die ein Level gelöst haben). Sie ist an das StatisticsViewModel gebunden und lädt Daten dynamisch aus der Datenbank über DatabaseService. Beide Tabs teilen sich dasselbe StatisticsViewModel und greifen auf dessen Properties (available_players, available_levels, player_statistics, level_statistics) zu. Die Trennung in zwei Tabs ermöglicht unterschiedliche Perspektiven auf dieselben Spielabschluss-Daten: spieler-zentriert vs. level-zentriert. Die StatisticsView ist rein lesend und ermöglicht keine Manipulation der Daten. Alle angezeigten Werte stammen aus der player_completed_levels-Tabelle in SQLite und wurden beim Level-Completion über DatabaseService.save_completed_level() persistiert

UI-Komponenten:

- Hauptstruktur:
 - tab_widget (QTabWidget): - Container für beide Statistik-Tabs
 - player_tab (PlayerStatisticsTab): - Tab "Player Statistics"
 - level_tab (LevelStatisticsTab): - Tab "Level Statistics"
 - - back_btn (QPushButton): - Button "Back to Menu"
- PlayerStatisticsTab
 - player_combo (QComboBox): - Dropdown zur Spieler-Auswahl
 - results_table (QTableWidget): - Tabelle der Spielerstatistik
- LevelStatisticsTab
 - level_combo (QComboBox): - Dropdown zur Level-Auswahl
 - results_table (QTableWidget): - Tabelle der Levelstatistik

Abbildung 15: UML-Klasse StatisticsView



Quelle: Erstellt mit draw.io

3.3.5 Zusätzliche UI-Komponenten

Neben den vier Hauptviews (MainMenuView, LevelSelectionView, GameView, StatisticsView) existieren vier zusätzliche UI-Komponenten, die entweder technische Infrastruktur bereitstellen (MainWindow, SplashScreen) oder spezialisierte Rendering-Funktionen übernehmen (GameCanvas, CoordinateMapper).

GameCanvas:

Die GameCanvas-Klasse ist eine spezialisierte QGraphicsView-Subklasse, die als interaktives Spielfeld in der GameView fungiert. Sie erweitert QGraphicsView um Mouse-Event-Handling für click-basierte Bridge-Platzierung.

- Technische Details:
 - Erbt von: QGraphicsView
 - Rendert: QGraphicsScene mit GridPoints, Nodes und Bridges
 - Funktionalität: Überschreibt mousePressEvent() für Click-Handling
- Signals:
 - canvas_clicked_left (QPointF): Emittiert Szenen-Koordinaten bei Linksklick
 - canvas_clicked_right: Emittiert bei Rechtsklick (Abbruch-Placement)

Die GameCanvas empfängt Klicks vom Spieler, transformiert Pixel-Koordinaten in Szenen-Koordinaten via mapToScene() und emittiert canvas_clicked_left Signal. Die GameView nutzt anschließend den CoordinateMapper, um Szenen-Koordinaten in Grid-Koordinaten (Model-Ebene) zu konvertieren.

CoordinateMapper:

Der CoordinateMapper ist eine Utility-Klasse zur bidirektionalen Transformation zwischen Grid-Koordinaten (Model Layer) und Screen-Pixeln (View Layer). Er gehört technisch zum ViewModel-Layer, da er Präsentationslogik kapselt ohne UI-Komponenten zu enthalten.

Entkoppelt Spiellogik (arbeitet mit Grid-Koordinaten) von GUI-Rendering (arbeitet mit Pixeln). Ermöglicht einfache Anpassung der Darstellungsgröße ohne Änderungen am Model Layer.

- Konfiguration:
 - cell_size: 50px (Breite/Höhe einer Grid-Zelle)
 - offset_x: 20px (Linker Rand)
 - offset_y: 20px (Oberer Rand)

- Methoden:
 - `grid_point_to_screen()`: - Konvertiert GridPoint zu Pixeln
 - `screen_to_model()`: - Konvertiert Pixel zu Grid-Koordinaten
 - `screen_to_grid_point()`: - Findet GridPoint-Objekt an geklickter Position
 - `transform_path()`: - Konvertiert Bridge-Pfade für Rendering

MainWindow:

Das MainWindow ist die zentrale Fenstersteuerung der Anwendung und erbt von QMainWindow. Es verwaltet den kompletten Navigation-Flow zwischen allen Views über ein QStackedWidget und fungiert als Dependency-Injection-Container für ViewModels. Das MainWindow instanziiert beim Start den DatabaseService und injiziert ihn in alle ViewModels über deren Konstruktoren. Dies ermöglicht zentrale Verwaltung der Datenbankverbindung und erleichtert Testing durch Mock-Injection.

- Technische Details:
 - Erbt von: QMainWindow
 - Fenstergröße: Mindestens 800x600 Pixel
 - Icon: Lädt aus Icon aus den Ressourcen
 - Titel: "Network Architect"
- Komponenten:
 - `stack (QStackedWidget)`: - Container für alle Views
 - `db_service (DatabaseService)`: - Persistenzschicht für alle ViewModels

SplashScreen:

Der SplashScreen ist ein frameless Window, das während der Anwendungsinitialisierung (Datenbank-Setup, Asset-Loading) angezeigt wird. Nach Abschluss wechselt die Anwendung automatisch zum MainMenuView

- UI-Komponenten:
 - Logo: 90x90px QPixmap aus Ressourcen
 - Title: "Network Architect"
 - Version: "Version 1.0"
 - Status: Statisches Label "Initializing..."
- Technische Details:
 - Erbt von: QWidget
 - Fenstergröße: 400x300px (fixed)
 - Window-Flags: WindowStaysOnTopHint | FramelessWindowHint
 - Position: Zentriert auf Primärbildschirm

3.4 Persistence Layer

Der Persistence Layer wird durch den DatabaseService repräsentiert und ist für die dauerhafte Speicherung aller Spielstände, Level-Konfigurationen und Statistiken in SQLite verantwortlich. Er kapselt sämtliche Datenbankoperationen und stellt eine klare Schnittstelle zwischen der Geschäftslogik (Model Layer) und der physischen Datenspeicherung bereit.

Der DatabaseService implementiert das Repository-Pattern und entkoppelt die Anwendung von SQLite-spezifischen Details. Alle CRUD-Operationen für GameSession, Level, Player und Statistiken werden hier zentralisiert, wodurch die ViewModels und Models von Persistenz-Details befreit bleiben.

Verantwortlichkeiten:

- Initialisierung der SQLite-Datenbank beim Anwendungsstart
- CRUD-Operationen für GameSession, Level, Node, Bridge
- Laden und Speichern von Spielerstatistiken
- Transaktionsverwaltung zur Sicherstellung der Datenkonsistenz

Methoden:

- initialize_database() - Erstellt Datenbank-Schema falls nicht vorhanden
- get_all_players() - Gibt Liste aller Spieler zurück
- create_player() - Erstellt neuen Spieler
- get_player_by_id() - Lädt Spieler anhand ID
- get_unlocked_levels_by_player() - Gibt freigeschaltete Levels, eines Spielers, zurück
- create_level() - Erstellt neues Level in Datenbank
- get_level(level_id: int) - Lädt Level anhand ID
- get_all_levels() - Gibt Liste aller Levels zurück
- save_completed_level() - Speichert abgeschlossenes Level
- get_player_completed_levels() - Lädt Statistiken eines Spielers
- - get_level_completed_by_players() - Lädt Statistiken eines Levels

3.5 UML-Klassendiagramm MVVM

Das UML-Klassendiagramm zeigt die vollständige MVVM-Architektur von Network Architect mit drei Schichten: Model-Layer, ViewModel-Layer und View-Layer.

Model-Layer:

Die Geschäftsobjekte GameSession, Level, Network, Node und Bridge bilden die Spiellogik ab. Die Enumerationen NodeType und BridgeType definieren Typ-spezifische Eigenschaften, während NodeConfig die Startkonfiguration eines Levels speichert. Alle Beziehungen und Multiplizitäten entsprechen dem Spezifikationsdokument (siehe Kapitel 3.1.9).

ViewModel-Layer:

Drei ViewModels steuern die Hauptfunktionen der Anwendung:

- LevelSelectionMode: Spieler- und Levelauswahl, Spieler-Erstellung
- GameViewModel: Gameplay-Steuerung mit Bridge-Platzierung, Timer und Budget-Verwaltung
- StatisticsViewModel: Laden und Anzeigen von Spielstatistiken

Die ViewModels erben von QObject und nutzen PySide6's Signal/Slot-Mechanismus für reaktive View-Updates. Sie greifen über gerichtete Abhängigkeiten auf den Model-Layer zu, ohne selbst Geschäftsregeln zu enthalten.

View-Layer:

Vier Views bilden die Benutzeroberfläche:

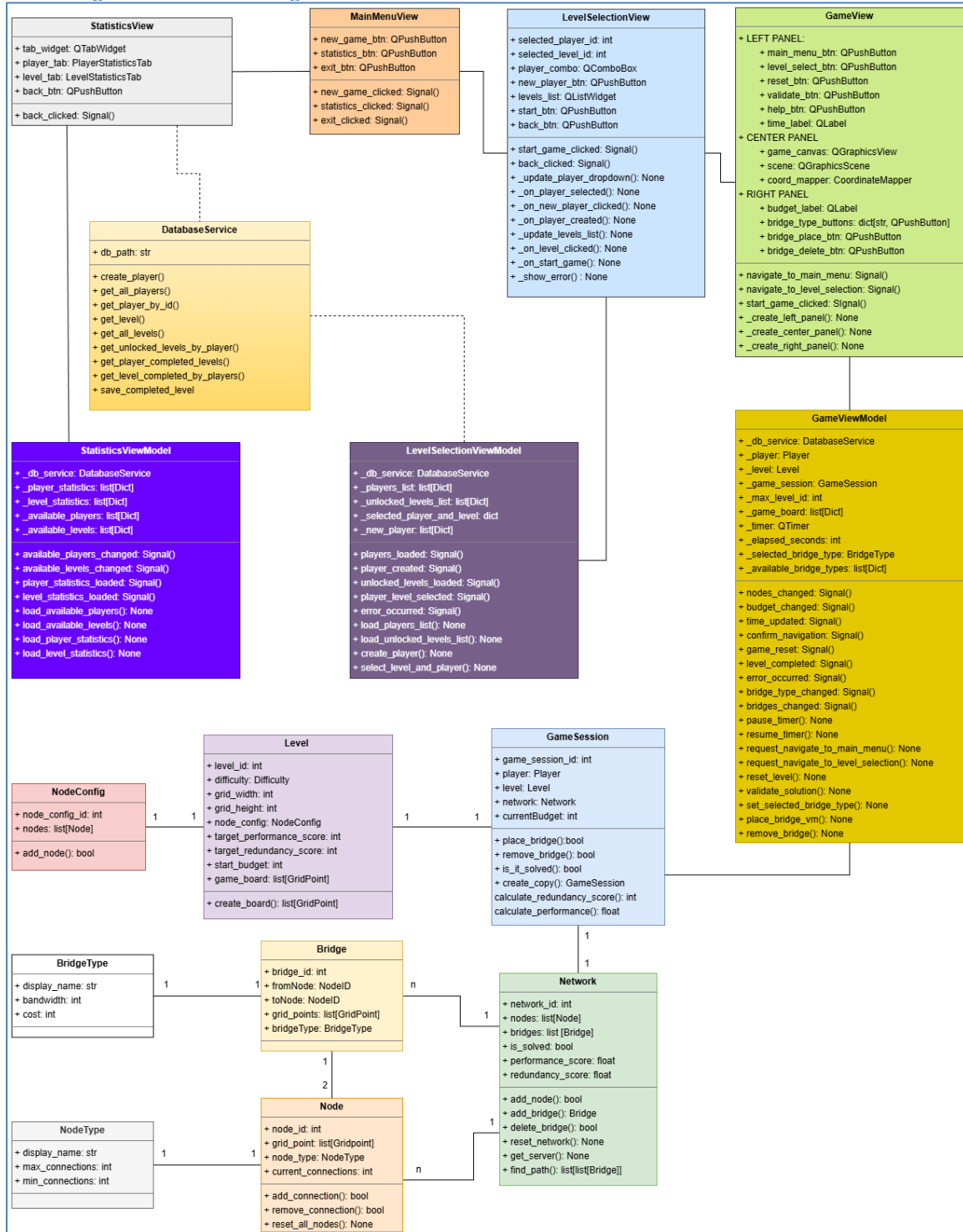
- MainMenuView: Startbildschirm ohne dediziertes ViewModel (nur Navigations-Signals)
- LevelSelectionMode: Gebunden an LevelSelectionMode
- GameView: Gebunden an GameViewModel mit 3-Panel-Layout
- StatisticsView: Gebunden an StatisticsViewModel mit Tab-Struktur

Alle Views erben von QWidget und kommunizieren ausschließlich über Signals mit ihren ViewModels bzw. mit MainWindow für Navigation.

Persistence-Layer:

Der DatabaseService bildet die Persistenzschicht und ist für SQLite-Datenbankoperationen verantwortlich. Er wird von allen ViewModels genutzt (über Dependency Injection im MainWindow) und stellt CRUD-Operationen für GameSession, Level, Player und Statistiken bereit. Der DatabaseService ist bewusst außerhalb des MVVM-Patterns angesiedelt, da er eine infrastrukturelle Komponente darstellt.

Abbildung 16: UML-Klassendiagramm MVVM



Quelle: Erstellt mit draw.io

4 Verhalten - Sequenzdiagramm Brücke platzieren

Das Kapitel Verhalten beschreibt den zeitlichen Ablauf zentraler Interaktionen in Network Architect anhand eines UML-Sequenzdiagramms. Am Beispiel des Szenarios "Brücke platzieren" wird gezeigt, wie Eingaben des Spielers über die GameView ins GameViewModel gelangen und dort an die Geschäftslogik im GameSession weitergeleitet werden, wo Budget- und Topologieregeln geprüft werden.

4.1 BridgeType-Auswahl

Der Spieler wählt den gewünschten Brückentyp aus der rechten Sidebar. Die GameView ruft `set_selected_bridge_type()` im GameViewModel auf, welches das Signal `bridge_type_changed()` emittiert. Die GameView markiert den Button daraufhin visuell.

4.2 FromNode, GridPoints und ToNode Auswahl

Der Spieler definiert den Verlauf der Bridge durch click-basierte Interaktion:

1. Klick auf Start-Node → GameView speichert `from_node_id`
2. Klick auf Grid-Points → GameView erweitert `grid_points_id`-Liste
3. Klick auf End-Node → GameView speichert `to_node_id`

Die GameView erzeugt daraus den Methodenaufruf `place_bridge_vm()` an das GameViewModel.

4.3 Bridgeplacement - Validierung und Platzierung

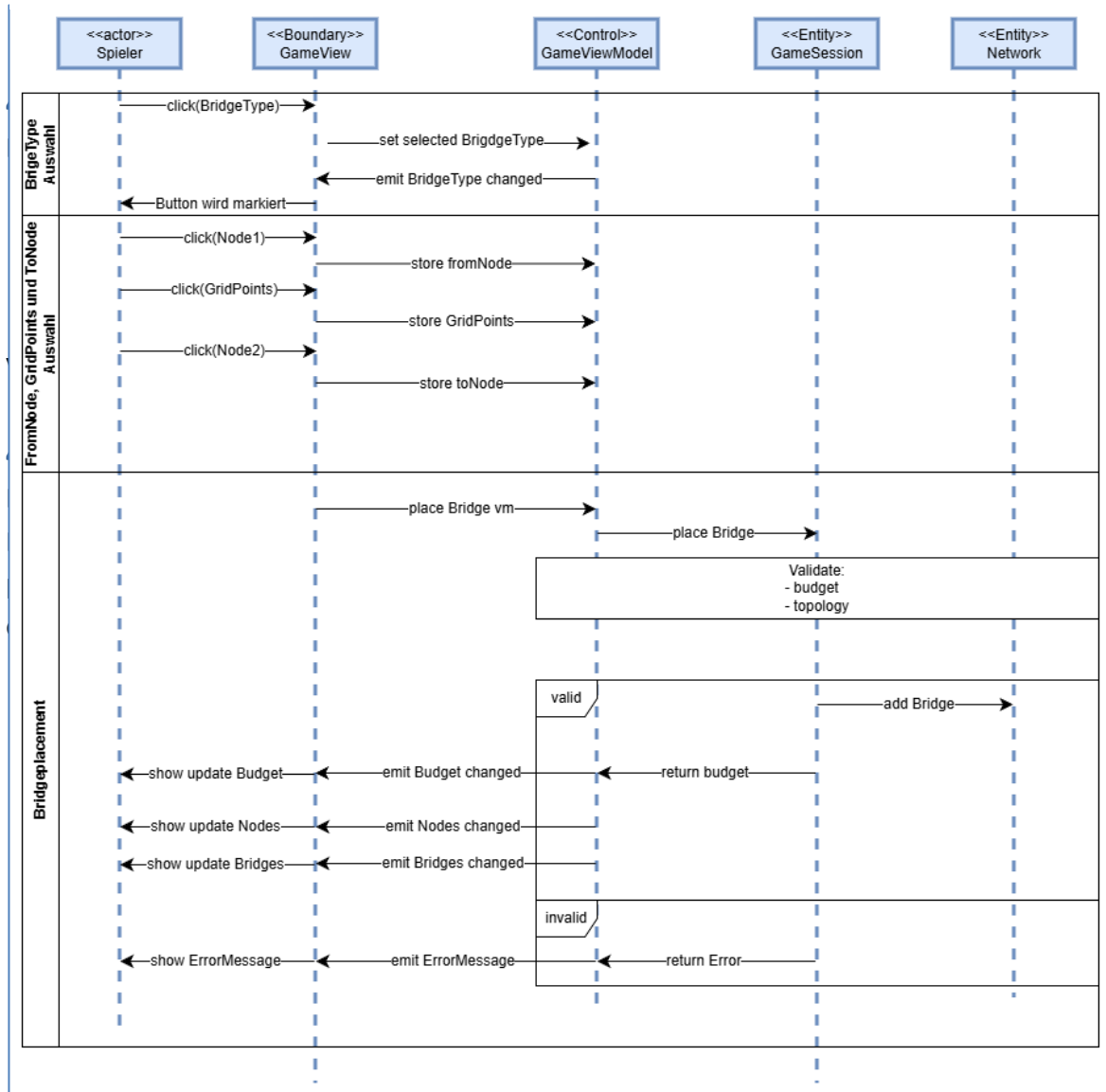
Die GameView ruft `place_bridge_vm()` auf, das GameViewModel leitet die Anfrage an GameSession weiter. Die GameSession führt zwei zentrale Validierungen durch (Budget und Topologie):

- Budget-Validierung: Prüft, ob `current_budget >= bridge_cost`
- Topologie-Validierung: Prüft, ob Bridge-Kreuzungen vermieden werden, der Pfad geometrisch valid ist und beide Nodes freie Verbindungsslots haben

Valid-Fall: Bei erfolgreicher Validierung delegiert GameSession an Network (add Bridge), welches die Bridge erstellt und das aktualisierte Budget zurückgibt. Das GameViewModel aktualisiert die GameView durch drei Aufrufe: `update Budget`, `update Nodes` und `render Bridges`.

Invalid-Fall: Bei fehlgeschlagener Validierung gibt GameSession einen Fehler zurück. Das GameViewModel emittiert `error_occurred()`, die GameView zeigt eine QMessageBox mit der Fehlermeldung. Die Bridge wird nicht platziert und das Budget bleibt unverändert (GR17).

Abbildung 17: Sequenzdiagramm Brücke platzieren



Quelle: Erstellt mit draw.io

5 Verzeichnisse

5.1 Tabellenverzeichnis

Tabelle 1: NodeType-Eigenschaften	10
Tabelle 2: BridgeType-Eigenschaften	11

5.2 Abbildungsverzeichnis

Abbildung 1: UML-Klasse GameSession	6
Abbildung 2: UML-Klasse Level	7
Abbildung 3: UML-Klasse Network	8
Abbildung 4: UML-Klasse Node	9
Abbildung 5: UML-Klasse Bridge	9
Abbildung 6: UML-Klasse NodeType	10
Abbildung 7: UML-Klasse BridgeType	10
Abbildung 8: UML-Klasse NodeConfig	11
Abbildung 9: UML-Klasse LevelSelectionViewModel	12
Abbildung 10: UML-Klasse GameViewModel	14
Abbildung 11: UML-Klasse StatisticsViewModel	15
Abbildung 12: UML-Klasse MainMenuView	16
Abbildung 13: UML-Klasse LevelSelectionView	18
Abbildung 14: UML-Klasse GameView	20
Abbildung 15: UML-Klasse StatisticsView	21
Abbildung 16: UML-Klassendiagramm MVVM	27
Abbildung 17: Sequenzdiagramm Brücke platzieren	29